

Python 시각화 패키지 : 맷플롯리브(Matplotlib)

맷플롯리브(Matplotlib)는 파이썬에서 자료를 차트(chart)나 플롯(plot)으로 시각화하는 패키지이다. 맷플롯리브는 다음과 같은 정형화된 차트나 플롯 이외에도 저수준 API 를 사용한 다양한 시각화 기능을 제공한다.

- 라인 플롯(line plot)
- 스캐터 플롯(scatter plot)
- 컨투어 플롯(contour plot)
- 서피스 플롯(surface plot)
- 바 차트(bar chart)
- 히스토그램(histogram)
- 박스 플롯(box plot)

맷플롯리브를 사용한 시각화 예제들을 보고 싶다면 맷플롯리브 갤러리 웹사이트를 방문한다.

- <http://matplotlib.org/gallery.html>

pyplot 서브패키지

맷플롯리브 패키지에는 pyplot 라는 서브패키지가 존재한다. 이 pyplot 서브패키지는 매트랩(matlab) 이라는 수치해석 소프트웨어의 시각화 명령을 거의 그대로 사용할 수 있도록 맷플롯리브의 하위 API 를 포장(wrapping)한 명령어 집합을 제공한다.

간단한 시각화 프로그램을 만드는 경우에는 pyplot 서브패키지의 명령만으로도 충분하다. 다음에 설명할 명령어들도 별도의 설명이 없으면 pyplot 패키지의 명령어라고 생각하면 된다.

맷플롯리브 패키지를 사용할 때는 보통 다음과 같이 주 패키지는 mpl 이라는 별칭(alias)으로 임포트하고 pyplot 서브패키지는 plt 라는 다른 별칭으로 임포트하여 사용하는 것이 관례이므로 그대로 사용한다.

```
import matplotlib as mpl
import matplotlib.pyplot as plt
```

%matplotlib

주피터 노트북을 사용하는 경우에는 다음처럼 %matplotlib 매직(magic) 명령으로 노트북 내부에 그림을 표시하도록 지정해야 한다.

```
%matplotlib inline
```

라인 플롯

plot

가장 간단한 플롯은 선을 그리는 라인 플롯(line plot)이다. 라인 플롯은 데이터가 시간, 순서 등에 따라 어떻게 변화하는지 보여주기 위해 사용한다.

명령은 pyplot 서브패키지의 plot 명령을 사용한다.

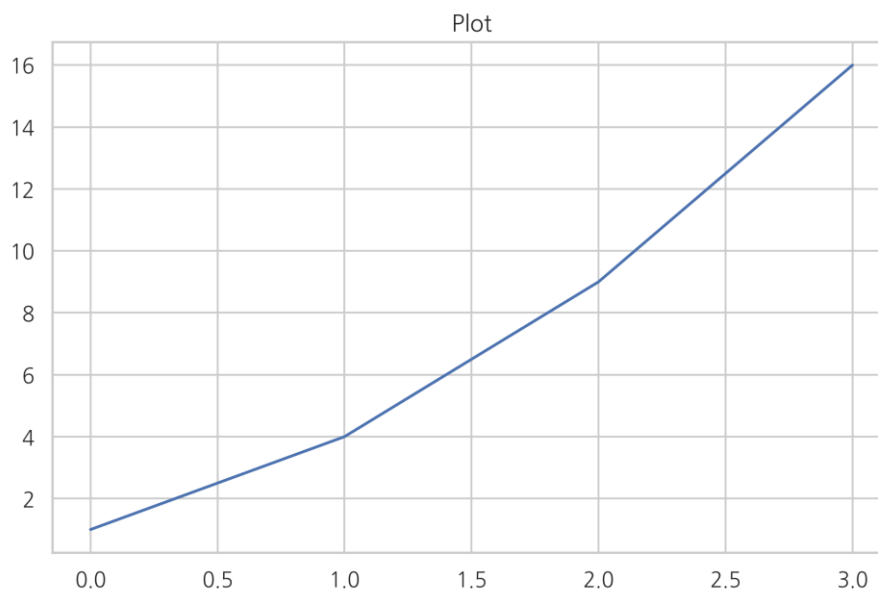
- http://matplotlib.org/api/pyplot_api.html#matplotlib.pyplot.plot

만약 데이터가 1, 4, 9, 16 으로 변화하였다면 다음과 같이 plot 명령에 데이터 리스트 혹은 ndarray 객체를 넘긴다.

```
plt.title("Plot")
```

```
plt.plot([1, 4, 9, 16])
```

```
plt.show()
```



이 때 x 축의 자료 위치 즉, 틱(tick)은 자동으로 0, 1, 2, 3 이 된다. 만약 이 x tick 위치를 별도로 명시하고 싶다면 다음과 같이 두 개의 같은 길이의 리스트 혹은 배열 자료를 넣는다.

title()

title 함수는 제목을 표시한다.

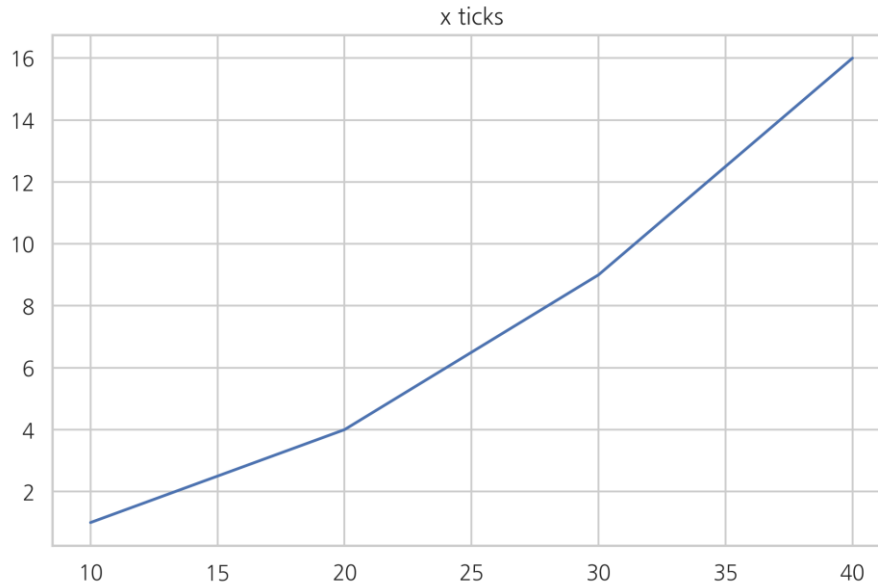
Show()

show 함수는 시각화 명령을 실제로 차트로 렌더링(rendering)하고 마우스 움직임 등의 이벤트를 기다리라는 지시이다. 주피터 노트북에서는 셀 단위로 플롯 명령을 자동 렌더링 해주므로 show 명령이 필요없지만 일반 파이썬 인터프리터로 가동되는 경우를 대비하여 항상 마지막에 실행하도록 한다. show 명령을 주면 마지막 플롯 명령으로부터 반환된 플롯 객체의 표현도 가려주는 효과가 있다.

```
plt.title("x ticks")
```

```
plt.plot([10, 20, 30, 40], [1, 4, 9, 16])
```

```
plt.show()
```



한글폰트 사용

맷플롯리브에서 한글을 사용하려면 다음과 같이 한글 폰트를 적용해야 한다. 당연히 해당 폰트는 컴퓨터에 설치되어 있어야 한다.

나눔고딕 폰트를 사용하고자 한다면, 나눔고딕 폰트 설치법은 다음과 같다.

- 윈도우/맥
 - <http://hangeul.naver.com/2017/nanum> 에서 폰트 인스톨러를 내려받아 실행한다.
- 리눅스
 - 콘솔에서 다음과 같이 실행한다.

```
sudo apt install -y fonts-nanum*
sudo fc-cache -fv
rm ~/.cache/matplotlib -rf
```

폰트를 설치한 후에는 다음 명령으로 원하는 폰트가 설치되어 있는지 확인한다.

```
import matplotlib.font_manager

matplotlib.font_manager._rebuild()
sorted([f.name for f in matplotlib.font_manager.fontManager.ttflist if
f.name.startswith("Nanum")])
```

```
['Nanum Brush Script',
 'Nanum Pen Script',
 'NanumBarunGothic',
 'NanumBarunGothic',
 'NanumBarunGothic',
 'NanumBarunGothic',
 'NanumBarunpen',
 'NanumBarunpen',
 'NanumGothic',
 'NanumGothic',
 'NanumGothic',
 'NanumGothic',
 'NanumGothic Eco',
 'NanumGothic Eco',
 'NanumGothic Eco',
 'NanumGothicCoding',
 'NanumGothicCoding',
```

```
'NanumMyeongjo',  
'NanumMyeongjo',  
'NanumMyeongjo',  
'NanumMyeongjo Eco',  
'NanumMyeongjo Eco',  
'NanumMyeongjo Eco',  
'NanumSquare',  
'NanumSquare',  
'NanumSquare',  
'NanumSquare',  
'NanumSquareRound',  
'NanumSquareRound',  
'NanumSquareRound',  
'NanumSquareRound']
```

설치된 폰트를 사용하는 방법은 크게 두가지 방법이 있다.

- rc parameter 설정으로 이후의 그림 전체에 적용하는 방법
- 인수를 사용하여 개별 텍스트 관련 명령에만 적용하는 방법

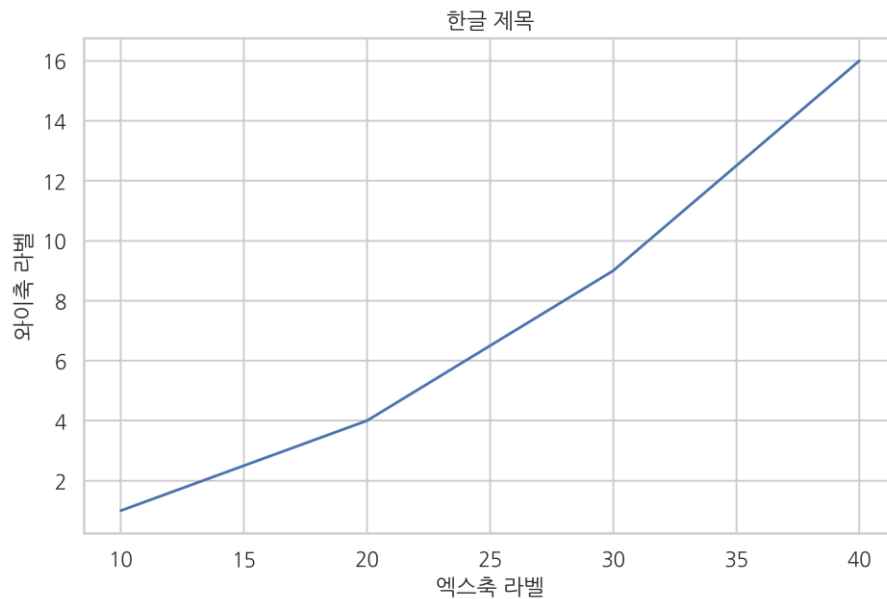
한글 문자열은 항상 유니코드를 사용해야 한다.

첫번째 방법 : rc parameter 를 설정하여 이후의 그림 전체에 적용해 보자

```
# 폰트 설정  
mpl.rc('font', family='NanumGothic')  
# 유니코드에서 음수 부호설정  
mpl.rc('axes', unicode_minus=False)
```

정상적으로 폰트가 설치되고 rc parameter 가 제대로 설정되었다면 다음 코드를 실행하였을 때 한글이 잘 보여야 한다.

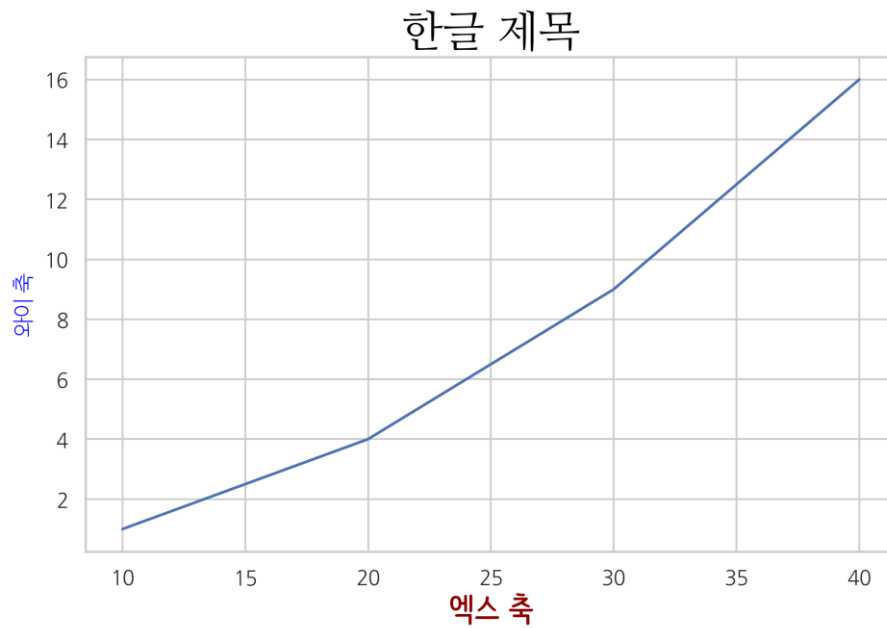
```
plt.title('한글 제목')  
plt.plot([10, 20, 30, 40], [1, 4, 9, 16])  
plt.xlabel("엑스축 라벨")  
plt.ylabel("와이축 라벨")  
plt.show()
```



만약 객체마다 별도의 폰트를 적용하고 싶을 때는 다음과 같이 폰트 패밀리, 색상, 크기를 정하여 플롯 명령의 fontdict 인수에 넣는다.

```
font1 = {'family': 'NanumMyeongjo', 'size': 24,  
         'color': 'black'}  
font2 = {'family': 'NanumBarunpen', 'size': 18, 'weight': 'bold',  
         'color': 'darkred'}  
font3 = {'family': 'NanumBarunGothic', 'size': 12, 'weight': 'light',  
         'color': 'blue'}
```

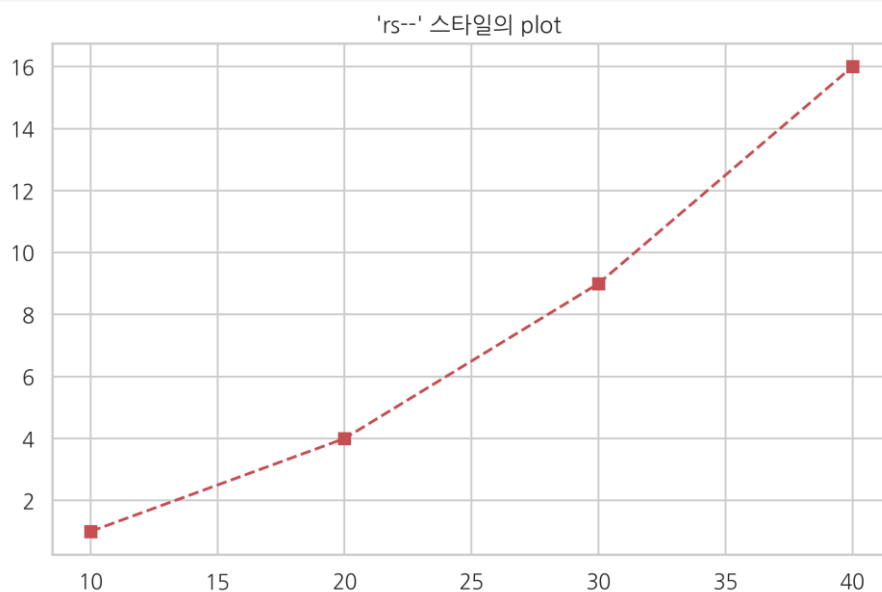
```
plt.plot([10, 20, 30, 40], [1, 4, 9, 16])  
plt.title('한글 제목', fontdict=font1)  
plt.xlabel('엑스 축', fontdict=font2)  
plt.ylabel('와이 축', fontdict=font3)  
plt.show()
```



스타일 지정

플롯 명령어는 보는 사람이 그림을 더 알아보기 쉽게 하기 위해 다양한 스타일(style)을 지원한다. plot 명령어에서는 다음과 같이 추가 문자열 인수를 사용하여 스타일을 지원한다.

```
plt.title("'rs--' 스타일의 plot ")  
plt.plot([10, 20, 30, 40], [1, 4, 9, 16], 'rs--')  
plt.show()
```



스타일 문자열은 색깔(color), 마커(marker), 선 종류(line style)의 순서로 지정한다. 만약 이 중 일부가 생략되면 디폴트값이 적용된다.

색상 (color)

색상을 지정하는 방법은 색 이름 혹은 약자를 사용하거나 # 문자로 시작되는 RGB16 진수 코드를 사용한다.

자주 사용되는 색깔은 한글자 약자를 사용할 수 있으며 약자는 아래 표에 정리하였다. 전체 색깔 목록은 다음 웹사이트를 참조한다.

- http://matplotlib.org/examples/color/named_colors.html

문자열	약자
blue	b
green	g
red	r
cyan	c
magenta	m
yellow	y
black	k
white	w

마커

데이터 위치를 나타내는 기호를 마커(marker)라고 한다.

마커의 종류는 다음과 같다.

마커 문자열	의미
.	point marker
,	pixel marker
o	circle marker
v	triangle_down marker
^	triangle_up marker
<	triangle_left marker
>	triangle_right marker

마커 문자열	의미
1	tri_down marker
2	tri_up marker
3	tri_left marker
4	tri_right marker
s	square marker
p	pentagon marker
*	star marker
h	hexagon1 marker
H	hexagon2 marker
+	plus marker
x	x marker
D	diamond marker
d	thin_diamond marker

선 스타일

선 스타일에는 실선(solid), 대시선(dashed), 점선(dotted), 대시-점선(dash-dot) 이 있다. 지정 문자열은 다음과 같다.

선 스타일 문자열	의미
-	solid line style
--	dashed line style
-.	dash-dot line style
:	dotted line style

기타 스타일

라인 플롯에서는 앞서 설명한 세 가지 스타일 이외에도 여러가지 스타일을 지정할 수 있지만 이 경우에는 인수 이름을 정확하게 지정해야 한다.

사용할 수 있는 스타일 인수의 목록은 matplotlib.lines.Line2D 클래스에 대한 다음 웹사이트를 참조한다.

- http://matplotlib.org/api/lines_api.html#matplotlib.lines.Line2D

라인 플롯에서 자주 사용되는 기타 스타일은 다음과 같다.

스타일 문자열	약자	의미
color	c	선 색깔
linewidth	lw	선 굵기
linestyle	ls	선 스타일
marker		마커 종류
markersize	ms	마커 크기
markeredgecolor	mec	마커 선 색깔
markeredgewidth	mew	마커 선 굵기
markerfacecolor	mfc	마커 내부 색깔

```
plt.plot([10, 20, 30, 40], [1, 4, 9, 16], c="b",
         lw=5, ls="--", marker="o", ms=15, mec="g", mew=5, mfc="r")
plt.title("스타일 적용 예")
plt.show()
```

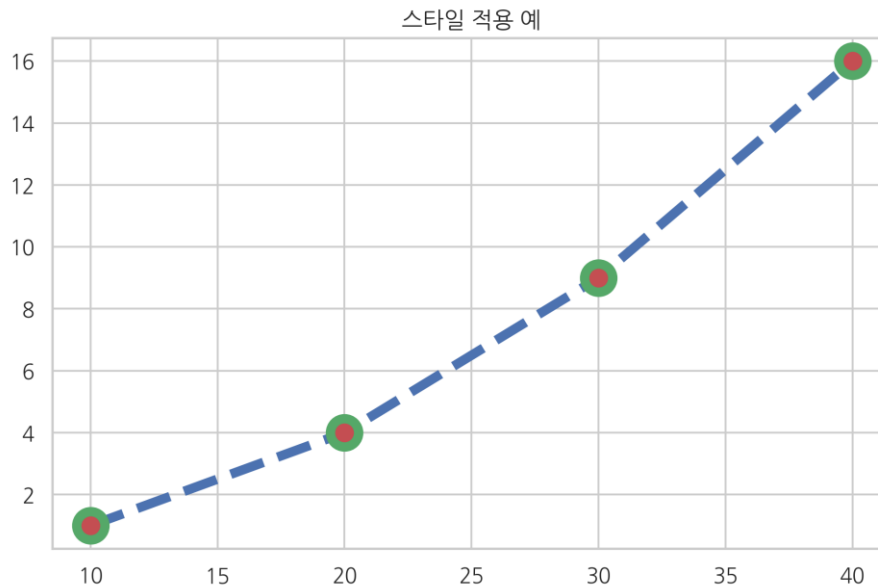


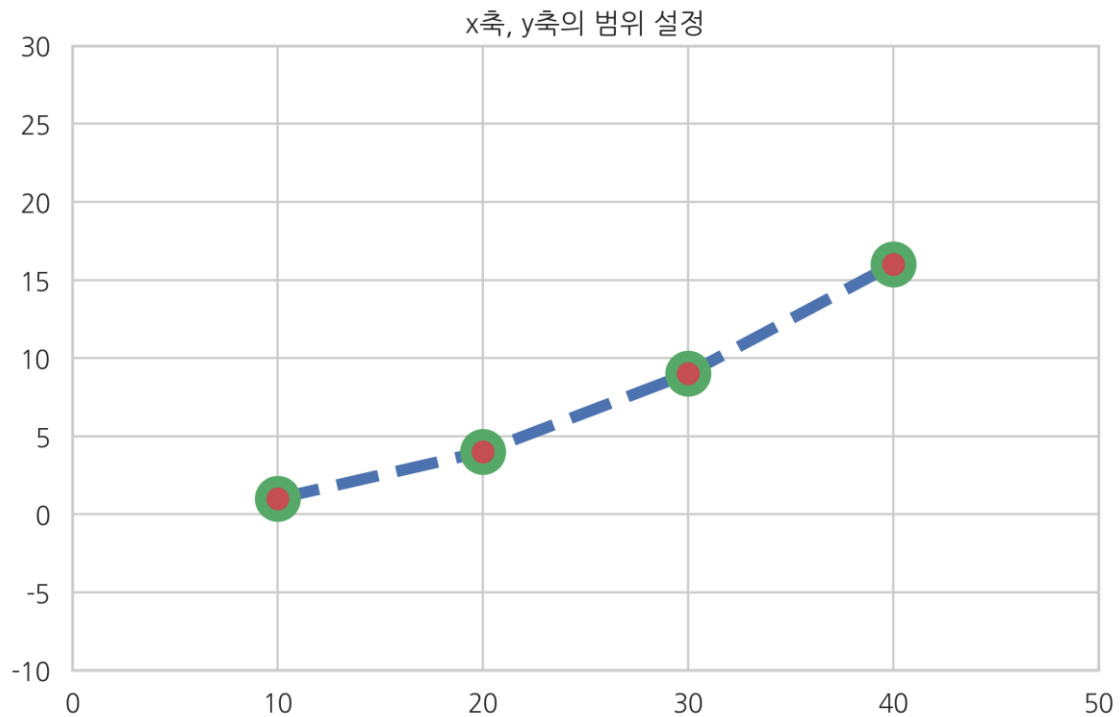
그림 범위 지정

xlim

ylim

플롯 그림을 보면 몇몇 점들은 그림의 범위 경계선에 있어서 잘 보이지 않는 경우가 있을 수 있다. 그림의 범위를 수동으로 지정하려면 xlim 명령과 ylim 명령을 사용한다. 이 명령들은 그림의 범위가 되는 x 축, y 축의 최소값과 최대값을 지정한다.

```
plt.title("x 축, y 축의 범위 설정")
plt.plot([10, 20, 30, 40], [1, 4, 9, 16],
         c="b", lw=5, ls="--", marker="o", ms=15, mec="g", mew=5, mfc="r")
plt.xlim(0, 50)
plt.ylim(-10, 30)
plt.show()
```



틱 설정

Xticks / yticks

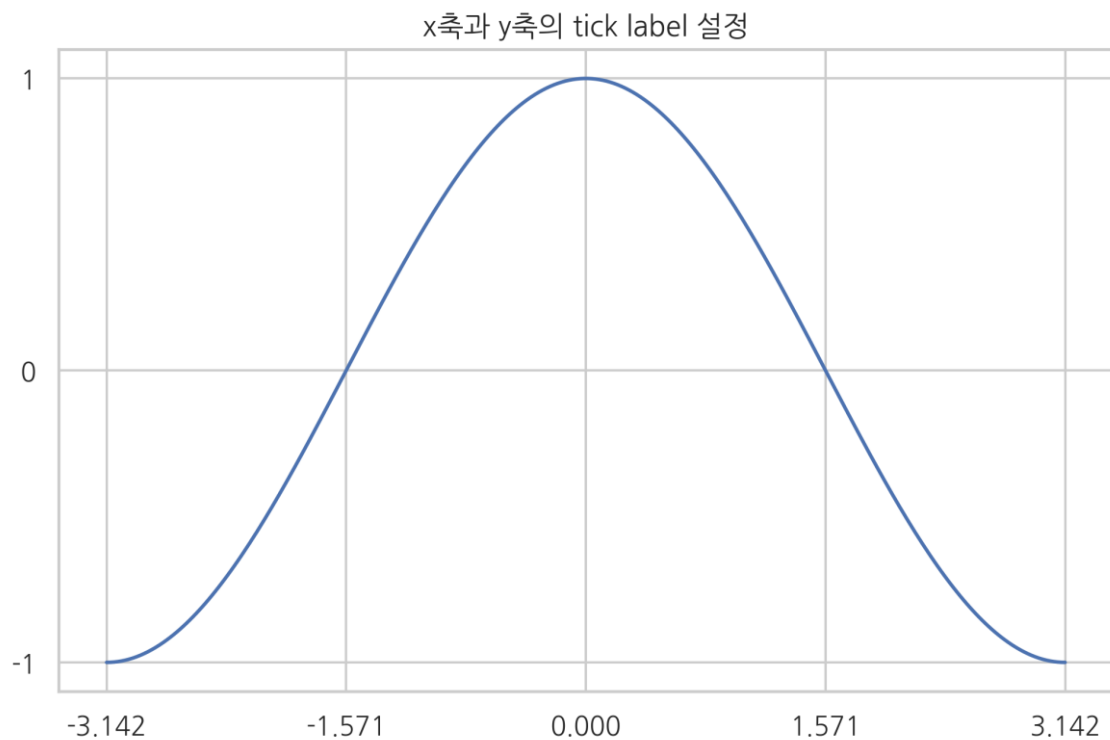
플롯이나 차트에서 축상의 위치 표시 지점을 틱(tick)이라고 하고 이 틱에 써진 숫자 혹은 글자를 틱 라벨(tick label)이라고 한다. 틱의 위치나 틱 라벨은 맷플롯리브가 자동으로 정해주지만 만약 수동으로 설정하고 싶다면 xticks 명령이나 yticks 명령을 사용한다.

```
X = np.linspace(-np.pi, np.pi, 256)
```

```

C = np.cos(X)
plt.title("x 축과 y 축의 tick label 설정")
plt.plot(X, C)
plt.xticks([-np.pi, -np.pi / 2, 0, np.pi / 2, np.pi])
plt.yticks([-1, 0, +1])
plt.show()

```

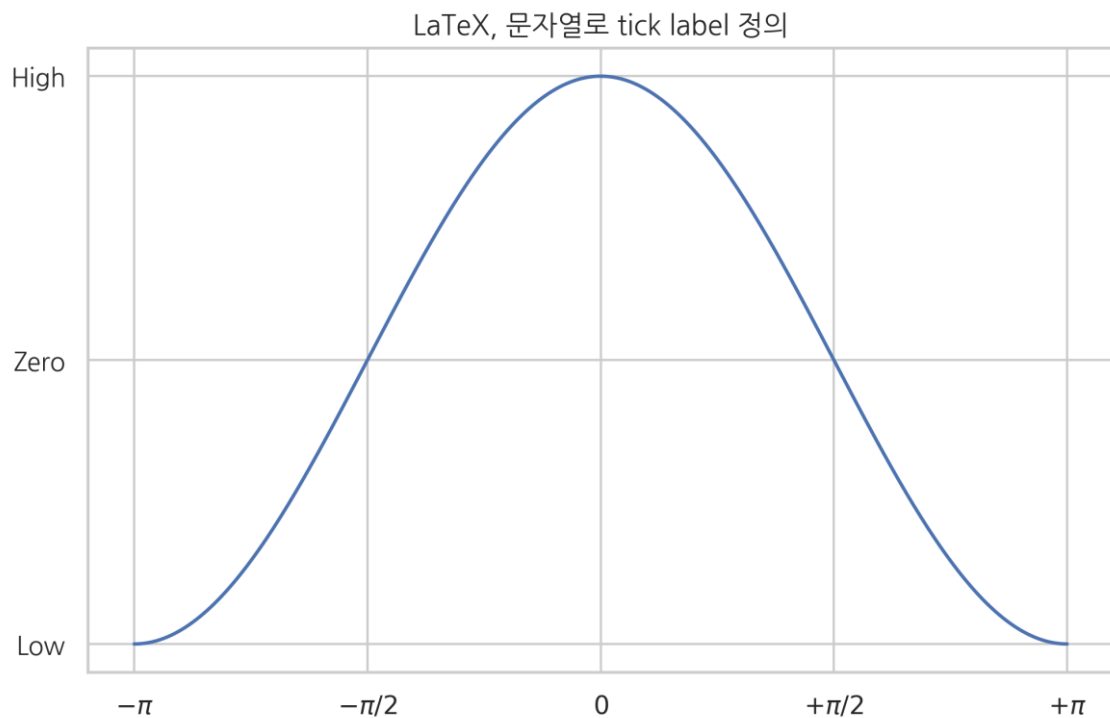


틱 라벨 문자열에는 \$\$ 사이에 LaTeX 수학 문자식을 넣을 수도 있다.

```

X = np.linspace(-np.pi, np.pi, 256)
C = np.cos(X)
plt.title("LaTeX, 문자열로 tick label 정의")
plt.plot(X, C)
plt.xticks([-np.pi, -np.pi / 2, 0, np.pi / 2, np.pi],
            [r'$-\pi$', r'$-\pi/2$', r'$0$', r'$+\pi/2$', r'$+\pi$'])
plt.yticks([-1, 0, 1], ["Low", "Zero", "High"])
plt.show()

```

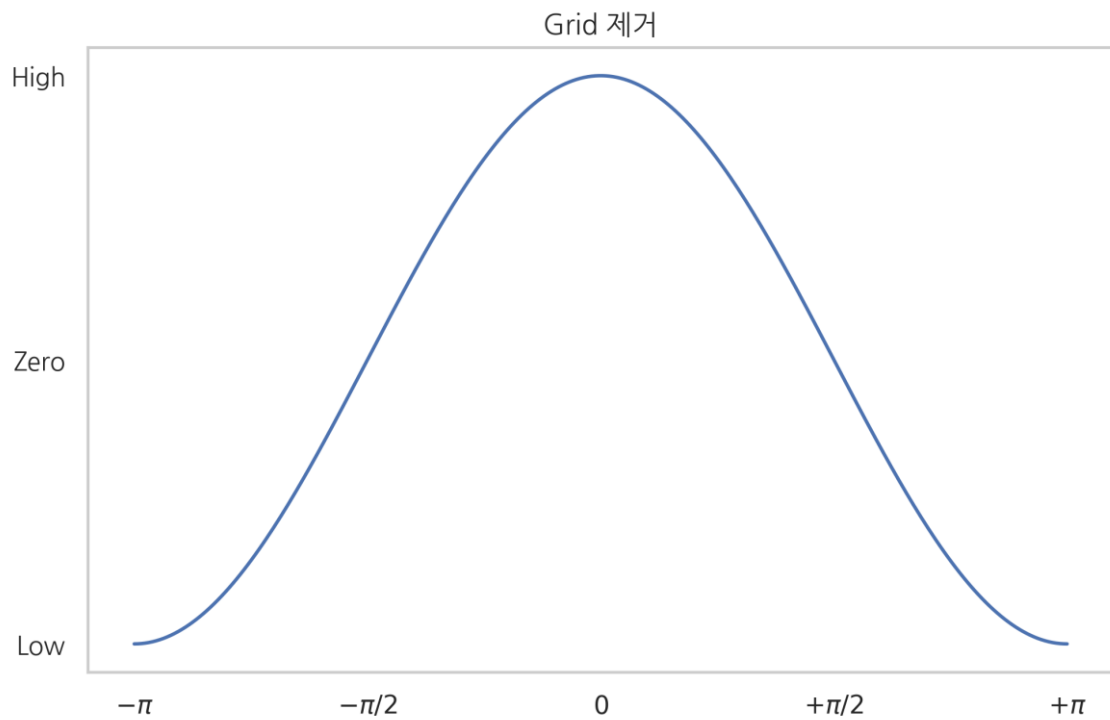


그리드 설정

grid

위 그림을 보면 틱 위치를 잘 보여주기 위해 그림 중간에 그리드 선(grid line)이 자동으로 그려진 것을 알 수 있다. 그리드를 사용하지 않으려면 `grid(False)` 명령을 사용한다. 다시 그리드를 사용하려면 `grid(True)`를 사용한다.

```
X = np.linspace(-np.pi, np.pi, 256)
C = np.cos(X)
plt.title("Grid 제거")
plt.plot(X, C)
plt.xticks([-np.pi, -np.pi / 2, 0, np.pi / 2, np.pi],
           [r'$-\pi$', r'$-\pi/2$', r'$0$', r'$+\pi/2$', r'$+\pi$'])
plt.yticks([-1, 0, 1], ["Low", "Zero", "High"])
plt.grid(False)
plt.show()
```

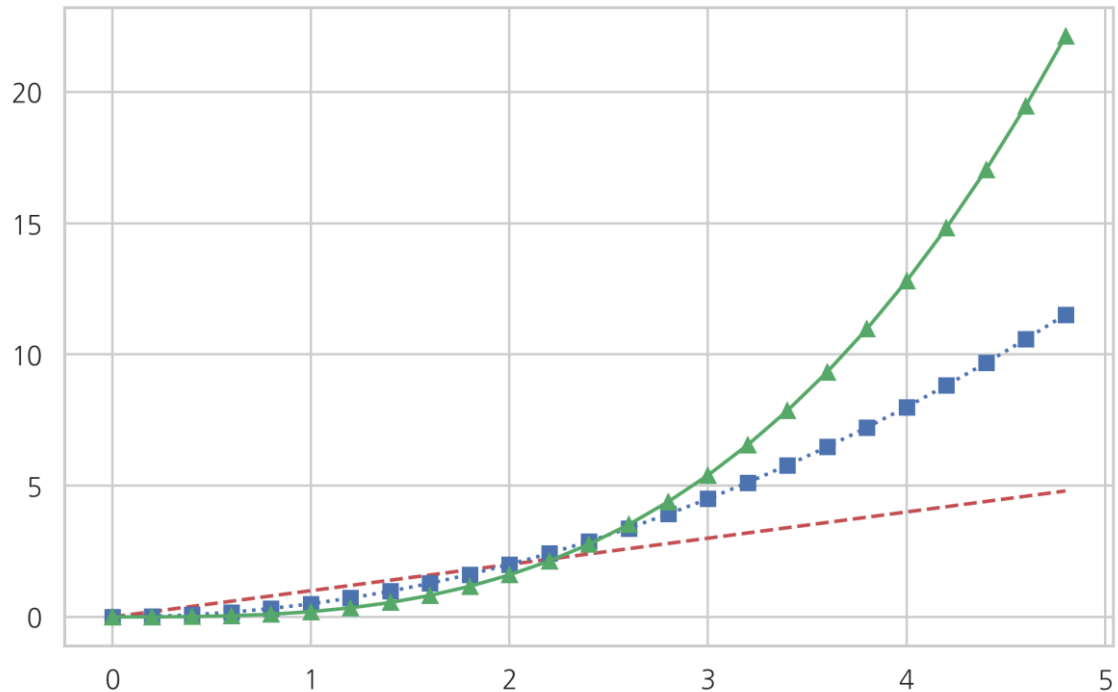


여러개의 선을 그리기

라인 플롯에서 선을 하나가 아니라 여러개를 그리고 싶은 경우에는 x 데이터, y 데이터, 스타일 문자열을 반복하여 인수로 넘긴다. 이 경우에는 하나의 선을 그릴 때처럼 x 데이터나 스타일 문자열을 생략할 수 없다.

```
t = np.arange(0., 5., 0.2)
plt.title("라인 플롯에서 여러개의 선 그리기")
plt.plot(t, t, 'r--', t, 0.5 * t**2, 'bs:', t, 0.2 * t**3, 'g^-')
plt.show()
```

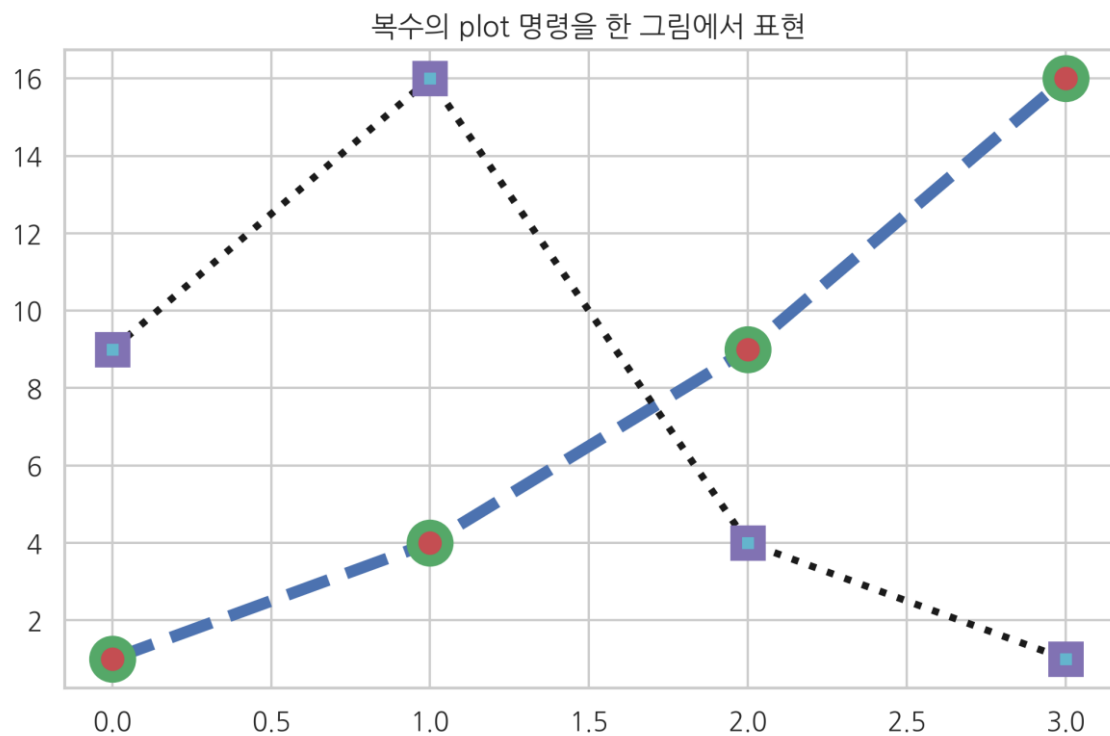
라인 플롯에서 여러개의 선 그리기



겹쳐 그리기

하나의 plot 명령이 아니라 복수의 plot 명령을 하나의 그림에 겹쳐서 그릴 수도 있다.

```
plt.title("복수의 plot 명령을 한 그림에서 표현")
plt.plot([1, 4, 9, 16],
         c="b", lw=5, ls="--", marker="o", ms=15, mec="g", mew=5, mfc="r")
# plt.hold(True) # <- 1.5 버전에서는 이 코드가 필요하다.
plt.plot([9, 16, 4, 1],
         c="k", lw=3, ls=":", marker="s", ms=10, mec="m", mew=5, mfc="c")
# plt.hold(False) # <- 1.5 버전에서는 이 코드가 필요하다.
plt.show()
```



범례

legend

여러 개의 라인 플롯을 동시에 그리는 경우에는 각 선이 무슨 자료를 표시하는지를 보여주기 위해 legend 명령으로 범례(legend)를 추가할 수 있다. 범례의 위치는 자동으로 정해지지만 수동으로 설정하고 싶으면 loc 인수를 사용한다. 인수에는 문자열 혹은 숫자가 들어가며 가능한 코드는 다음과 같다.

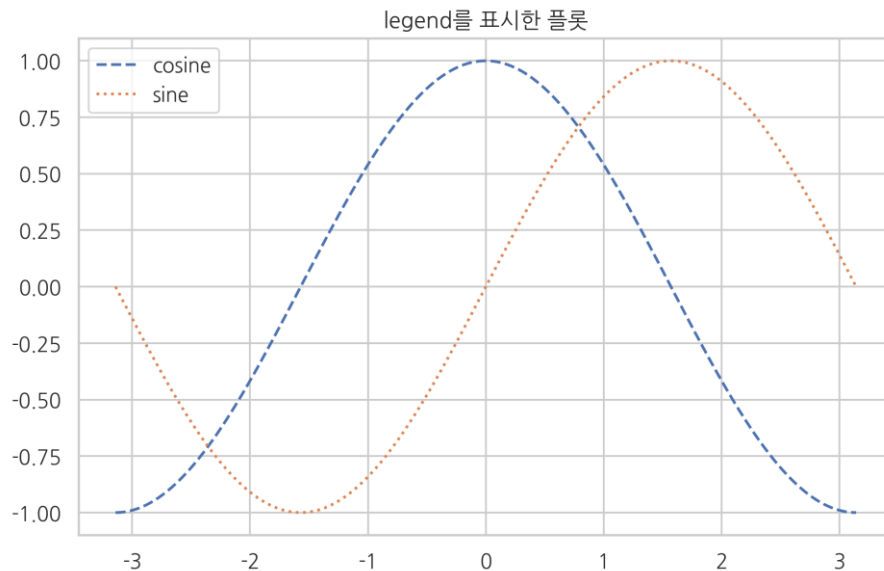
loc 문자열	숫자
best	0
upper right	1
upper left	2
lower left	3
lower right	4
right	5
center left	6
center right	7
lower center	8

loc 문자열	숫자
upper center	9
center	10

```

X = np.linspace(-np.pi, np.pi, 256)
C, S = np.cos(X), np.sin(X)
plt.title("legend 를 표시한 플롯")
plt.plot(X, C, ls="--", label="cosine")
plt.plot(X, S, ls=":", label="sine")
plt.legend(loc=2)
plt.show()

```



x 축, y 축 라벨, 타이틀

xlabel / ylabel

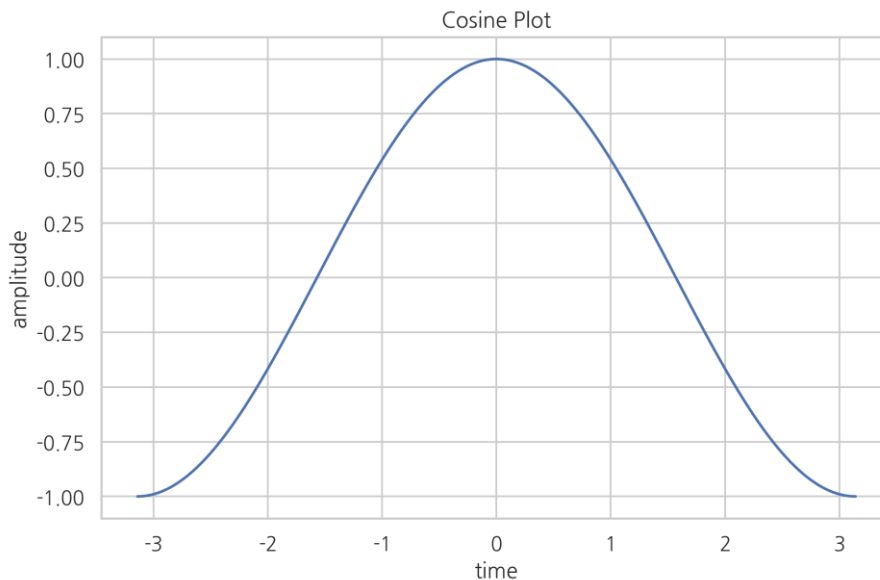
플롯의 x 축 위치와 y 축 위치에는 각각 그 데이터가 의미하는 바를 표시하기 위해 라벨(label)을 추가할 수 있다. 라벨을 붙이려면 xlabel, ylabel 명령을 사용한다. 또 플롯의 위에는 title 명령으로 제목(title)을 붙일 수 있다.

```

X = np.linspace(-np.pi, np.pi, 256)
C, S = np.cos(X), np.sin(X)
plt.plot(X, C, label="cosine")

```

```
plt.xlabel("time")
plt.ylabel("amplitude")
plt.title("Cosine Plot")
plt.show()
```



그림의 구조

맷플롯리브가 그리는 그림은 Figure 객체, Axes 객체, Axis 객체 등으로 구성된다.

Figure 객체는 한 개 이상의 Axes 객체를 포함하고 Axes 객체는 다시 두 개 이상의 Axis 객체를 포함한다.

Figure 는 그림이 그려지는 캔버스나 종이를 뜻하고 Axes 는 하나의 플롯, 그리고 Axis 는 가로축이나 세로축 등의 축을 뜻한다. Axes 와 Axis 의 철자에 주의한다.

Figure 객체

모든 그림은 Figure 객체이다. 정식으로는 matplotlib.figure.Figure 클래스 객체에 포함되어 있다. 내부 플롯(inline plot)이 아닌 경우에는 하나의 Figure 는 하나의 아이디 숫자와 윈도우(Window)를 가진다. 주피터 노트북에서는 윈도우 객체가 생성되지 않지만 파이썬을 독립 실행하는 경우에는 하나의 Figure 당 하나의 윈도우를 별도로 가진다.

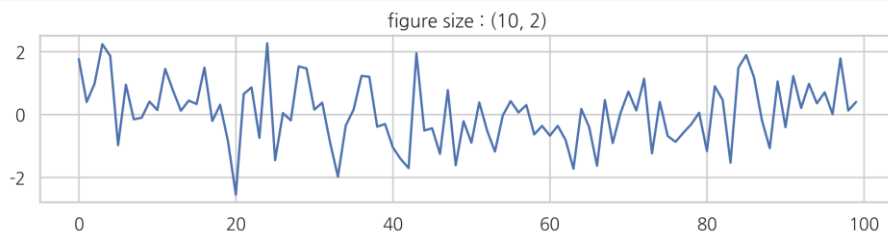
Figure 객체에 대한 자세한 설명은 다음 웹사이트를 참조한다.

- http://matplotlib.org/api/figure_api.html#matplotlib.figure.Figure

figure

원래 Figure 를 생성하려면 figure 명령을 사용하여 그 반환값으로 Figure 객체를 얻어야 한다. 그러나 일반적인 plot 명령 등을 실행하면 자동으로 Figure 를 생성해주기 때문에 일반적으로는 figure 명령을 잘 사용하지 않는다. figure 명령을 명시적으로 사용하는 경우는 여러 개의 윈도우를 동시에 띄워야 하거나 (line plot 이 아닌 경우), Jupyter 노트북 등에서 (line plot 의 경우) 그림의 크기를 설정하고 싶을 때이다. 그림의 크기는 figsize 인수로 설정한다.

```
np.random.seed(0)
f1 = plt.figure(figsize=(10, 2))
plt.title("figure size : (10, 2)")
plt.plot(np.random.randn(100))
plt.show()
```



gcf

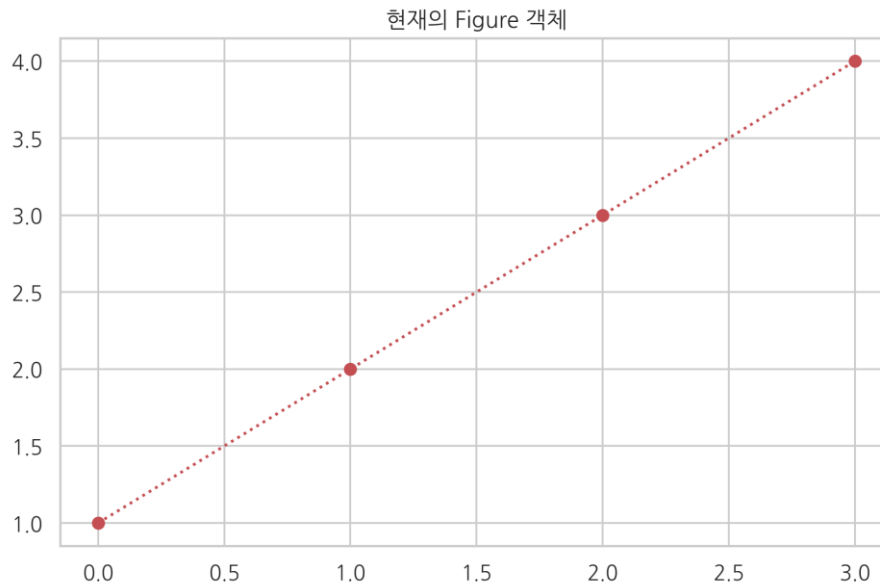
현재 사용하고 있는 Figure 객체를 얻으려면 gcf 명령을 사용한다.

```
f1 = plt.figure(1)
plt.title("현재의 Figure 객체")
plt.plot([1, 2, 3, 4], 'ro:')

f2 = plt.gcf()
print(f1, id(f1))
print(f2, id(f2))
plt.show()
```

Figure(2400x1500) 140676427789072

Figure(2400x1500) 140676427789072



Axes 객체와 subplot 명령

때로는 다음과 같이 하나의 윈도우(Figure)안에 여러 개의 플롯을 배열 형태로 보여야하는 경우도 있다. Figure 안에 있는 각각의 플롯은 Axes 라고 불리는 객체에 속한다.

Axes 객체에 대한 자세한 설명은 다음 웹사이트를 참조한다.

- http://matplotlib.org/api/axes_api.html#matplotlib.axes.Axes

subplot

Figure 안에 Axes 를 생성하려면 원래 subplot 명령을 사용하여 명시적으로 Axes 객체를 얻어야 한다. 그러나 plot 명령을 바로 사용해도 자동으로 Axes 를 생성해 준다.

subplot 명령은 그리드(grid) 형태의 Axes 객체들을 생성하는데 Figure 가 행렬(matrix)이고 Axes 가 행렬의 원소라고 생각하면 된다.

예를 들어 위와 아래 두 개의 플롯이 있는 경우 행이 2 이고 열이 1 인 2 x 1 행렬이다. subplot 명령은 세개의 인수를 가지는데 처음 두개의 원소가 전체 그리드 행렬의 모양을 지시하는 두 숫자이고 세번째 인수가 네 개 중 어느 것인지를 의미하는 숫자이다. 따라서 위/아래 두개의 플롯을 하나의 Figure 안에 그리려면 다음처럼 명령을 실행해야 한다. 여기에서 숫자 인덱싱은 파이썬이 아닌 Matlab 관행을 따르기 때문에 첫번째 플롯을 가리키는 숫자가 0 이 아니라 1 임에 주의하라.

```
subplot(2, 1, 1)
```

```
# 여기에서 윗부분에 그릴 플롯 명령 실행
```

```
subplot(2, 1, 2)
```

```
# 여기에서 아랫부분에 그릴 플롯 명령 실행
```

tight_layout

tight_layout 명령을 실행하면 플롯간의 간격을 자동으로 맞춰준다.

```
x1 = np.linspace(0.0, 5.0)
```

```
x2 = np.linspace(0.0, 2.0)
```

```
y1 = np.cos(2 * np.pi * x1) * np.exp(-x1)
```

```
y2 = np.cos(2 * np.pi * x2)
```

```
ax1 = plt.subplot(2, 1, 1)
```

```
plt.plot(x1, y1, 'yo-')
```

```
plt.title('A tale of 2 subplots')
```

```
plt.ylabel('Damped oscillation')
```

```
ax2 = plt.subplot(2, 1, 2)
```

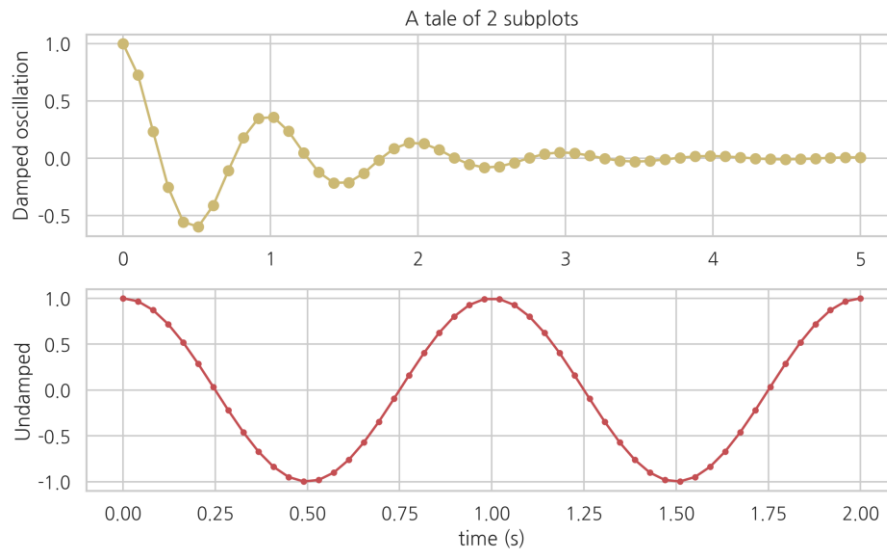
```
plt.plot(x2, y2, 'r.-')
```

```
plt.xlabel('time (s)')
```

```
plt.ylabel('Undamped')
```

```
plt.tight_layout()
```

```
plt.show()
```



만약 2 x 2 형태의 네 개의 플롯이라면 다음과 같이 그린다. 이 때 subplot 의 인수는 (2,2,1)를 줄여서 221 라는 하나의 숫자로 표시할 수도 있다. Axes 의 위치는 위에서 부터 아래로, 왼쪽에서 오른쪽으로 카운트한다.

```
np.random.seed(0)
```

```
plt.subplot(221)
```

```
plt.plot(np.random.rand(5))
```

```
plt.title("axes 1")
```

```
plt.subplot(222)
```

```
plt.plot(np.random.rand(5))
```

```
plt.title("axes 2")
```

```
plt.subplot(223)
```

```
plt.plot(np.random.rand(5))
```

```
plt.title("axes 3")
```

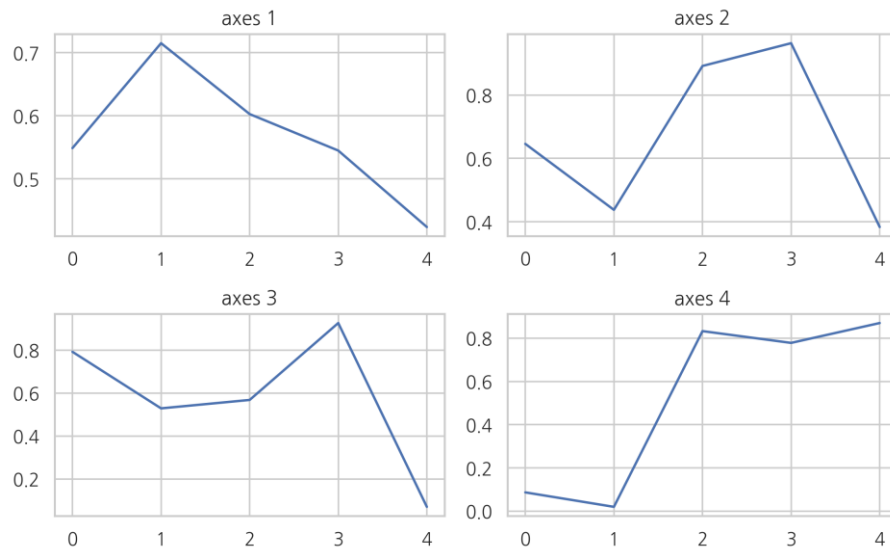
```
plt.subplot(224)
```

```
plt.plot(np.random.rand(5))
```

```
plt.title("axes 4")
```

```
plt.tight_layout()
```

```
plt.show()
```



subplots

subplots 명령으로 복수의 Axes 객체를 동시에 생성할 수도 있다. 이때는 2 차원 ndarray 형태로 Axes 객체가 반환된다.

```
fig, axes = plt.subplots(2, 2)
```

```
np.random.seed(0)
```

```
axes[0, 0].plot(np.random.rand(5))
```

```
axes[0, 0].set_title("axes 1")
```

```
axes[0, 1].plot(np.random.rand(5))
```

```
axes[0, 1].set_title("axes 2")
```

```
axes[1, 0].plot(np.random.rand(5))
```

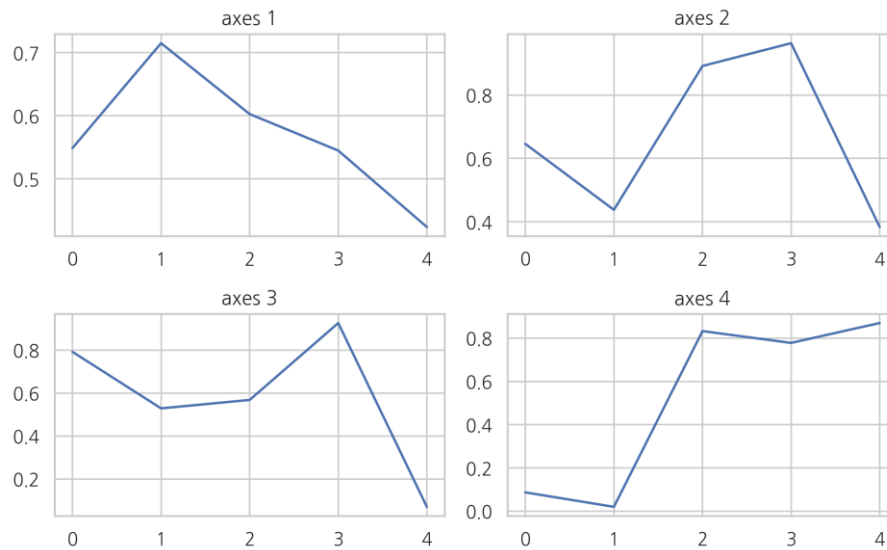
```
axes[1, 0].set_title("axes 3")
```

```
axes[1, 1].plot(np.random.rand(5))
```

```
axes[1, 1].set_title("axes 4")
```

```
plt.tight_layout()
```

```
plt.show()
```



Axis 객체와 축

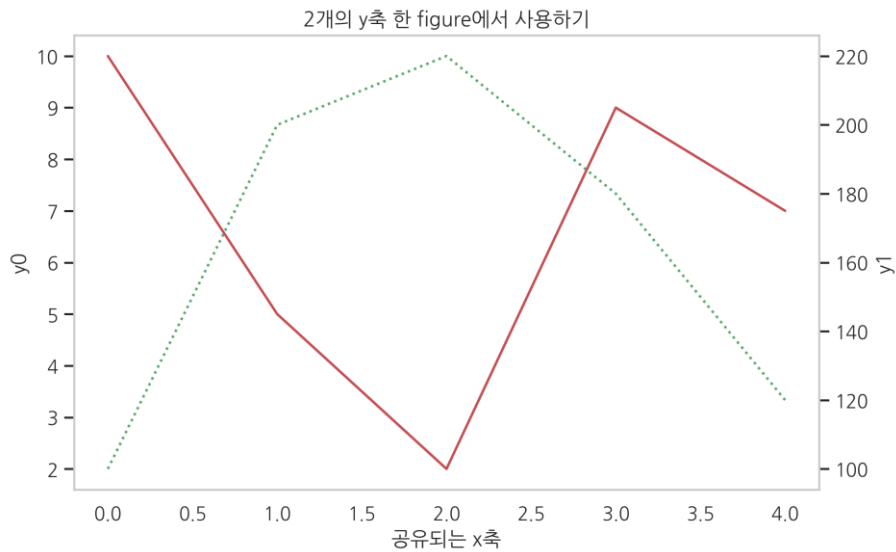
하나의 Axes 객체는 두 개 이상의 Axis 객체를 가진다. Axis 객체는 플롯의 가로축이나 세로축을 나타내는 객체이다. 보다 자세한 내용은 다음 웹사이트를 참조한다.

- https://matplotlib.org/api/axis_api.html

twinx

여러가지 플롯을 하나의 Axes 객체에 표시할 때 y 값의 크기가 달라서 표시하기 힘든 경우가 있다. 이 때는 다음처럼 twinx 명령으로 대해 복수의 y 축을 가진 플롯을 만들 수도 있다. twinx 명령은 x 축을 공유하는 새로운 Axes 객체를 만든다.

```
fig, ax0 = plt.subplots()
ax1 = ax0.twinx()
ax0.set_title("2 개의 y 축 한 figure 에서 사용하기")
ax0.plot([10, 5, 2, 9, 7], 'r-', label="y0")
ax0.set_ylabel("y0")
ax0.grid(False)
ax1.plot([100, 200, 220, 180, 120], 'g:', label="y1")
ax1.set_ylabel("y1")
ax1.grid(False)
ax0.set_xlabel("공유되는 x 축")
plt.show()
```

Matplotlib 의 여러가지 플롯

Matplotlib 는 기본적인 라인 플롯 이외에도 다양한 차트 / 플롯 유형을 지원한다.

바 차트

x 데이터가 카테고리 값인 경우에는 bar 명령과 barh 명령으로 바 차트(bar chart) 시각화를 할 수 있다. 가로 방향으로 바 차트를 그리려면 barh 명령을 사용한다. 자세한 내용은 다음 웹사이트를 참조한다.

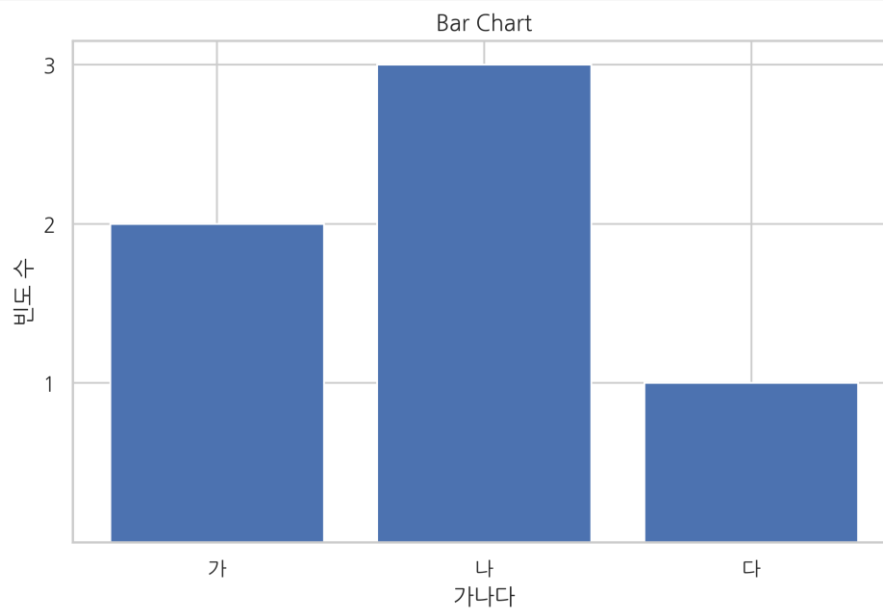
- http://matplotlib.org/api/pyplot_api.html#matplotlib.pyplot.bar
- http://matplotlib.org/api/pyplot_api.html#matplotlib.pyplot.barh

바 차트 작성시 주의점은 첫번째 인수인 left 가 x 축에서 바(bar)의 왼쪽 변의 위치를 나타낸다는 점이다.

```
import matplotlib as mpl
import matplotlib.pyplot as plt
```

```
y = [2, 3, 1]
x = np.arange(len(y))
xlabel = ['가', '나', '다']
plt.title("Bar Chart")
plt.bar(x, y)
```

```
plt.xticks(x, xlabel)
plt.yticks(sorted(y))
plt.xlabel("가나다")
plt.ylabel("빈도 수")
plt.show()
```



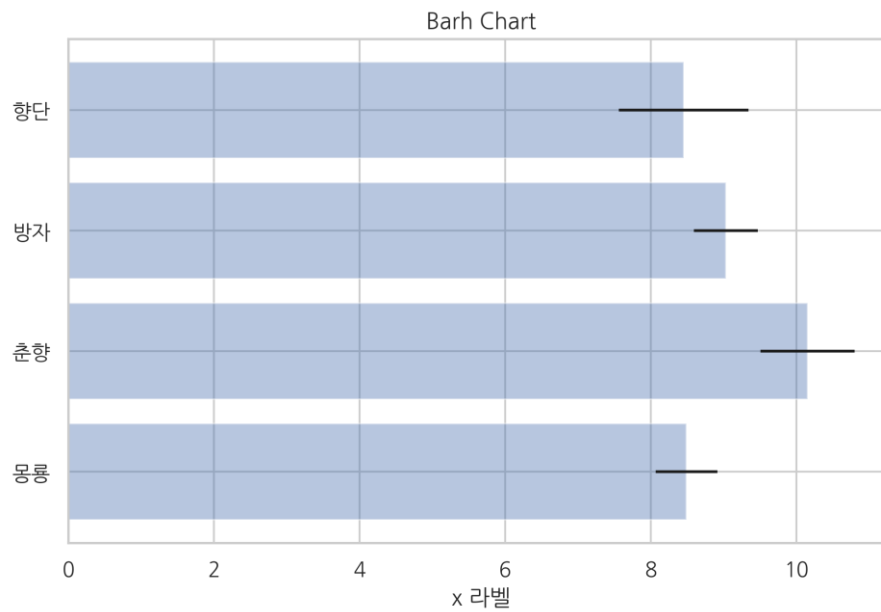
xerr 인수나 yerr 인수를 지정하면 에러 바(error bar)를 추가할 수 있다.

다음 코드에서 alpha 는 투명도를 지정한다. 0 이면 완전 투명, 1 이면 완전 불투명이다.

```
np.random.seed(0)

people = ['몽룡', '춘향', '방자', '향단']
y_pos = np.arange(len(people))
performance = 3 + 10 * np.random.rand(len(people))
error = np.random.rand(len(people))

plt.title("Barh Chart")
plt.barh(y_pos, performance, xerr=error, alpha=0.4)
plt.yticks(y_pos, people)
plt.xlabel('x 라벨')
plt.show()
```

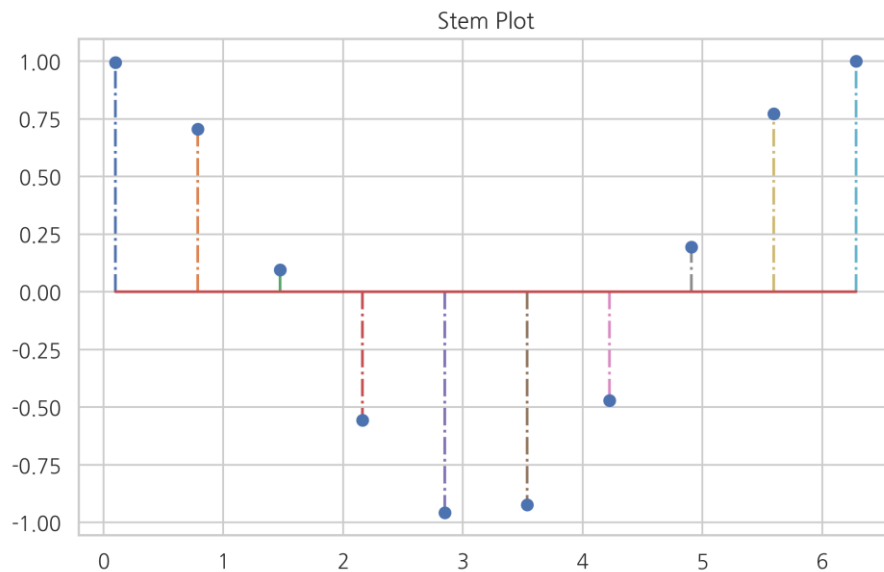


스텝 플롯

바 차트와 유사하지만 폭(width)이 없는 스텝 플롯(stem plot)도 있다. 주로 이산 확률 함수나 자기상관관계(auto-correlation)를 묘사할 때 사용된다.

- http://matplotlib.org/api/pyplot_api.html#matplotlib.pyplot.stem

```
x = np.linspace(0.1, 2 * np.pi, 10)
plt.title("Stem Plot")
plt.stem(x, np.cos(x), '-.')
plt.show()
```



파이 차트

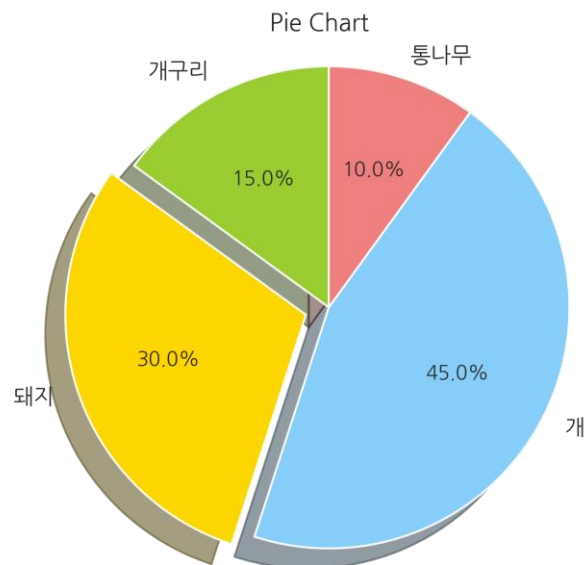
카테고리 별 값의 상대적인 비교를 해야 할 때는 pie 명령으로 파이 차트(pie chart)를 그릴 수 있다. 파이 차트를 그릴 때는 원의 형태를 유지할 수 있도록 다음 명령을 실행해야 한다.

```
plt.axis('equal')
```

자세한 내용은 다음을 참조한다.

- http://matplotlib.org/api/pyplot_api.html#matplotlib.pyplot.pie

```
labels = ['개구리', '돼지', '개', '통나무']
sizes = [15, 30, 45, 10]
colors = ['yellowgreen', 'gold', 'lightskyblue', 'lightcoral']
explode = (0, 0.1, 0, 0)
plt.title("Pie Chart")
plt.pie(sizes, explode=explode, labels=labels, colors=colors,
        autopct='%1.1f%%', shadow=True, startangle=90)
plt.axis('equal')
plt.show()
```

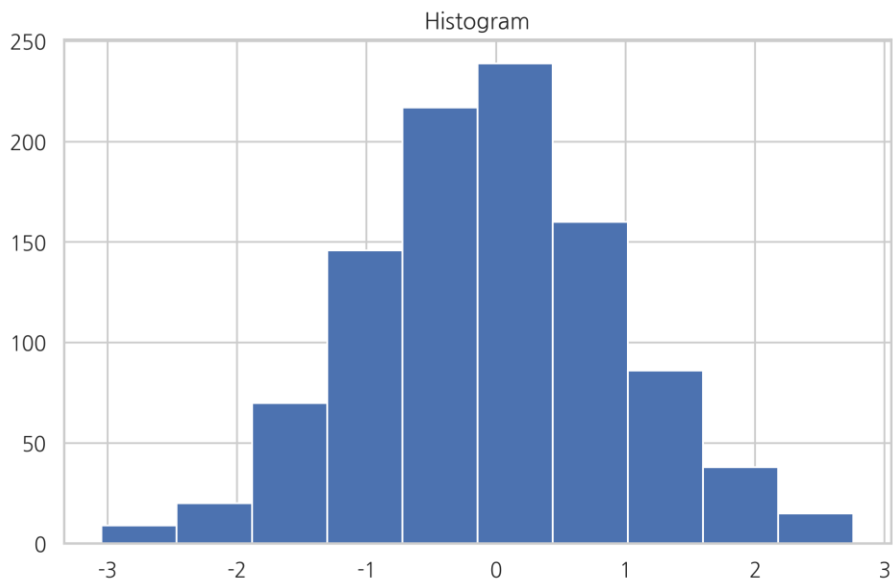


히스토그램

히스토그램을 그리기 위한 hist 명령도 있다. hist 명령은 bins 인수로 데이터를 집계할 구간 정보를 받는다. 반환값으로 데이터 집계 결과를 반환한다.

- http://matplotlib.org/api/pyplot_api.html#matplotlib.pyplot.hist

```
np.random.seed(0)
x = np.random.randn(1000)
plt.title("Histogram")
arrays, bins, patches = plt.hist(x, bins=10)
plt.show()
```



arrays

```
array([ 9., 20., 70., 146., 217., 239., 160., 86., 38., 15.])
```

bins

```
array([-3.04614305, -2.46559324, -1.88504342, -1.3044936 , -0.72394379,
       -0.14339397,  0.43715585,  1.01770566,  1.59825548,  2.1788053 ,
        2.75935511])
```

스캐터 플롯

2 차원 데이터 즉, 두 개의 실수 데이터 집합의 상관관계를 살펴보려면 scatter 명령으로 스캐터 플롯을 그린다. 스캐터 플롯의 점 하나의 위치는 데이터 하나의 x, y 값이다.

- http://matplotlib.org/api/pyplot_api.html#matplotlib.pyplot.scatter

```
np.random.seed(0)
```

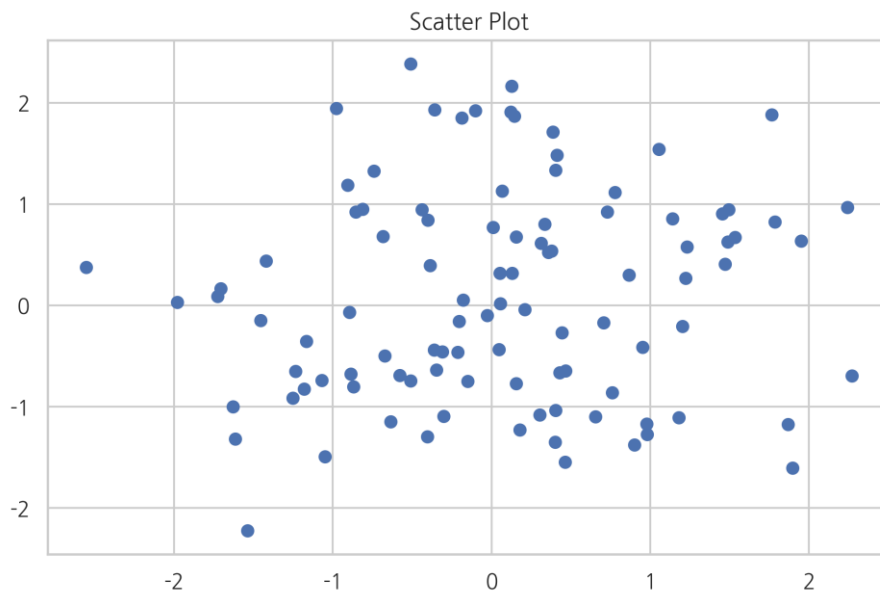
```
X = np.random.normal(0, 1, 100)
```

```
Y = np.random.normal(0, 1, 100)
```

```
plt.title("Scatter Plot")
```

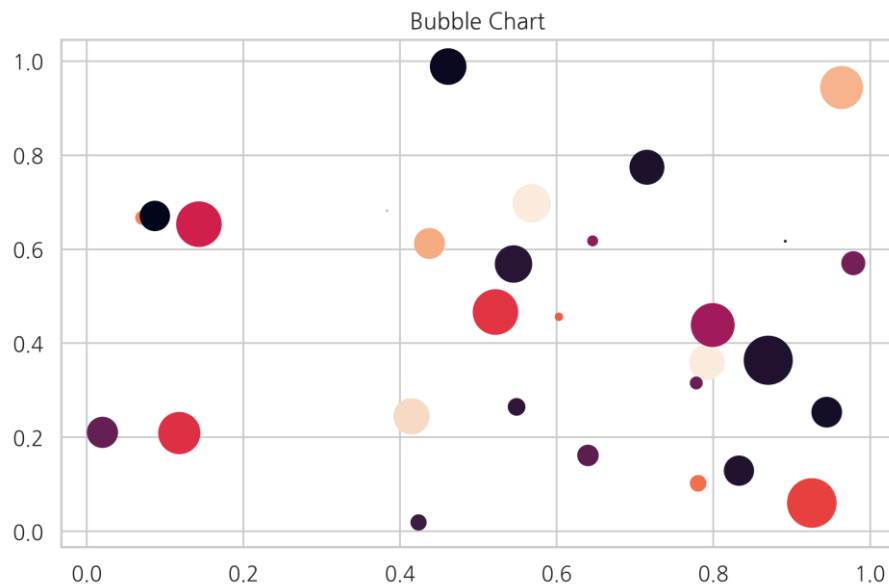
```
plt.scatter(X, Y)
```

```
plt.show()
```



데이터가 2 차원이 아니라 3 차원 혹은 4 차원인 경우에는 점 하나의 크기 혹은 색깔을 이용하여 다른 데이터 값을 나타낼 수도 있다. 이런 차트를 버블 차트(bubble chart)라고 한다. 크기는 s 인수로 색깔은 c 인수로 지정한다.

```
N = 30
np.random.seed(0)
x = np.random.rand(N)
y1 = np.random.rand(N)
y2 = np.random.rand(N)
y3 = np.pi * (15 * np.random.rand(N))**2
plt.title("Bubble Chart")
plt.scatter(x, y1, c=y2, s=y3)
plt.show()
```



Imshow

화상(image) 데이터처럼 행과 열을 가진 행렬 형태의 2 차원 데이터는 imshow 명령을 써서 2 차원 자료의 크기를 색깔로 표시하는 것이다.

- http://matplotlib.org/api/pyplot_api.html#matplotlib.pyplot.imshow

```
from sklearn.datasets import load_digits
digits = load_digits()
X = digits.images[0]
X

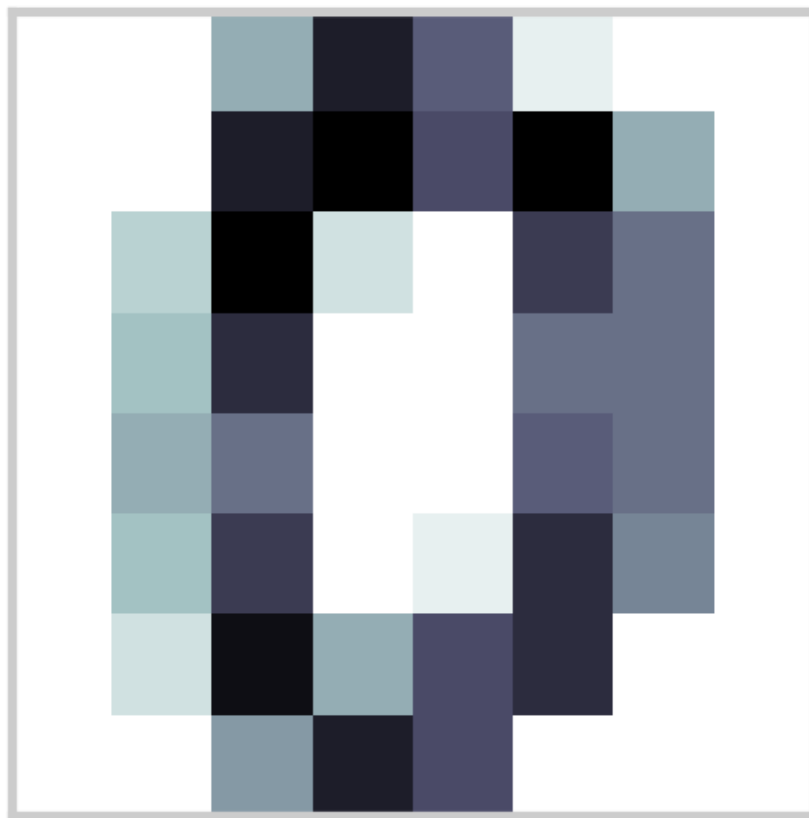
array([[ 0.,  0.,  5., 13.,  9.,  1.,  0.,  0.],
       [ 0.,  0., 13., 15., 10., 15.,  5.,  0.],
       [ 0.,  3., 15.,  2.,  0., 11.,  8.,  0.],
       [ 0.,  4., 12.,  0.,  0.,  8.,  8.,  0.],
       [ 0.,  5.,  8.,  0.,  0.,  9.,  8.,  0.],
       [ 0.,  4., 11.,  0.,  1., 12.,  7.,  0.],
       [ 0.,  2., 14.,  5., 10., 12.,  0.,  0.],
       [ 0.,  0.,  6., 13., 10.,  0.,  0.,  0.]])

plt.title("mnist digits; 0")
```



```
plt.imshow(X, interpolation='nearest', cmap=plt.cm.bone_r)
plt.xticks([])
plt.yticks([])
plt.grid(False)
plt.subplots_adjust(left=0.35, right=0.65, bottom=0.35, top=0.65)
plt.show()
```

mnist digits; 0



데이터 수치를 색으로 바꾸는 함수는 칼라맵(color map)이라고 한다. 칼라맵은 cmap 인수로 지정한다. 사용할 수 있는 칼라맵은 plt.cm 의 속성으로 포함되어 있다.

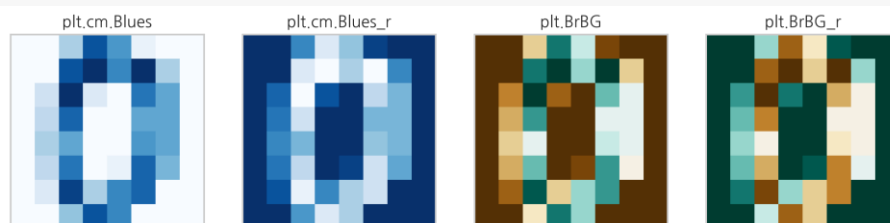
아래에 일부 칼라맵을 표시하였다. 칼라맵은 문자열로 지정해도 된다. 칼라맵에 대한 자세한 내용은 다음 웹사이트를 참조한다.

- <https://matplotlib.org/tutorials/colors/colormaps.html>

```
dir(plt.cm)[:10]
```

```
['Accent',  
 'Accent_r',  
 'Blues',  
 'Blues_r',  
 'BrBG',  
 'BrBG_r',  
 'BuGn',  
 'BuGn_r',  
 'BuPu',  
 'BuPu_r']
```

```
fig, axes = plt.subplots(1, 4, figsize=(12, 3),  
                          subplot_kw={'xticks': [], 'yticks': []})  
axes[0].set_title("plt.cm.Blues")  
axes[0].imshow(X, interpolation='nearest', cmap=plt.cm.Blues)  
axes[1].set_title("plt.cm.Blues_r")  
axes[1].imshow(X, interpolation='nearest', cmap=plt.cm.Blues_r)  
axes[2].set_title("plt.BrBG")  
axes[2].imshow(X, interpolation='nearest', cmap='BrBG')  
axes[3].set_title("plt.BrBG_r")  
axes[3].imshow(X, interpolation='nearest', cmap='BrBG_r')  
plt.show()
```



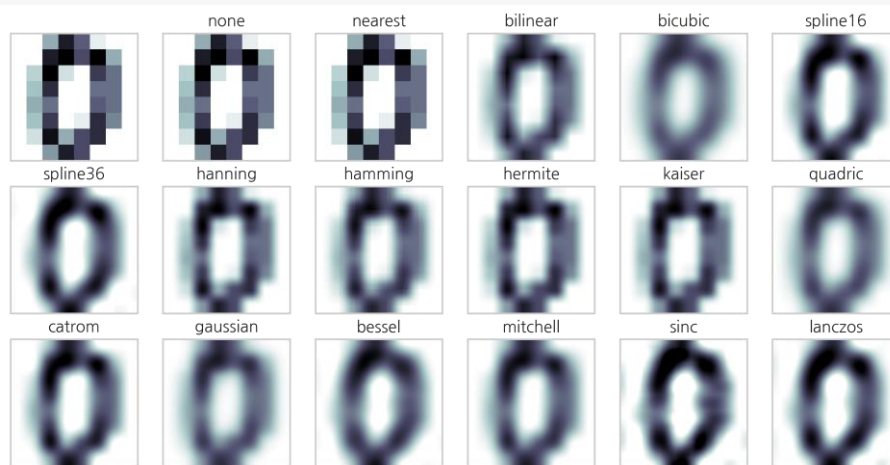
imshow 명령은 자료의 시각화를 돕기 위해 다양한 2 차원 인터폴레이션을 지원한다.

```
methods = [  
    None, 'none', 'nearest', 'bilinear', 'bicubic', 'spline16',  
    'spline36', 'hanning', 'hamming', 'hermite', 'kaiser', 'quadric',
```

```

'catrom', 'gaussian', 'bessel', 'mitchell', 'sinc', 'lanczos'
]
fig, axes = plt.subplots(3, 6, figsize=(12, 6),
                        subplot_kw={'xticks': [], 'yticks': []})
for ax, interp_method in zip(axes.flat, methods):
    ax.imshow(X, cmap=plt.cm.bone_r, interpolation=interp_method)
    ax.set_title(interp_method)
plt.show()

```



컨투어 플롯

입력 변수가 x, y 두 개이고 출력 변수가 z 하나인 경우에는 3 차원 자료가 된다. 3 차원 자료를 시각화하는 방법은 명암이 아닌 등고선(contour)을 사용하는 방법이다. contour 혹은 contourf 명령을 사용한다. contour 는 등고선만 표시하고 contourf 는 색깔로 표시한다. 입력 변수 x, y 는 그대로 사용할 수 없고 meshgrid 명령으로 그리드 포인트 행렬을 만들어야 한다. 더 자세한 내용은 다음 웹사이트를 참조한다.

- http://matplotlib.org/api/pyplot_api.html#matplotlib.pyplot.contour
- http://matplotlib.org/api/pyplot_api.html#matplotlib.pyplot.contourf

```

def f(x, y):
    return (1 - x / 2 + x ** 5 + y ** 3) * np.exp(-x ** 2 - y ** 2)

n = 256
x = np.linspace(-3, 3, n)

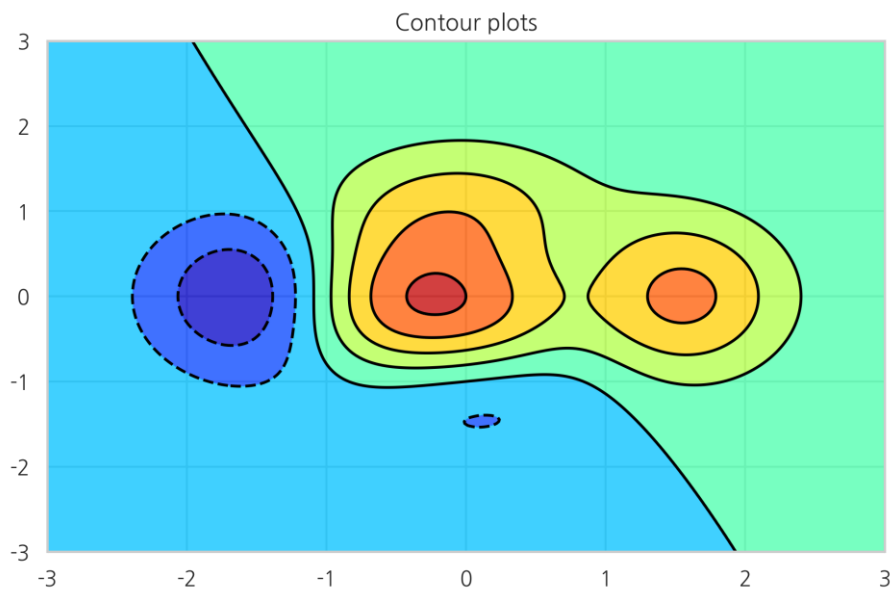
```

```

y = np.linspace(-3, 3, n)
XX, YY = np.meshgrid(x, y)
ZZ = f(XX, YY)

plt.title("Contour plots")
plt.contourf(XX, YY, ZZ, alpha=.75, cmap='jet')
plt.contour(XX, YY, ZZ, colors='black')
plt.show()

```



3D 서피스 플롯

3 차원 플롯은 등고선 플롯과 달리 Axes3D 라는 3 차원 전용 axes 를 생성하여 입체적으로 표시한다.

plot_wireframe, plot_surface 명령을 사용한다.

- http://matplotlib.org/mpl_toolkits/mplot3d/api.html#mpl_toolkits.mplot3d.axes3d.Axes3D.plot_wireframe
- http://matplotlib.org/mpl_toolkits/mplot3d/api.html#mpl_toolkits.mplot3d.axes3d.Axes3D.plot_surface

```

from mpl_toolkits.mplot3d import Axes3D
X = np.arange(-4, 4, 0.25)

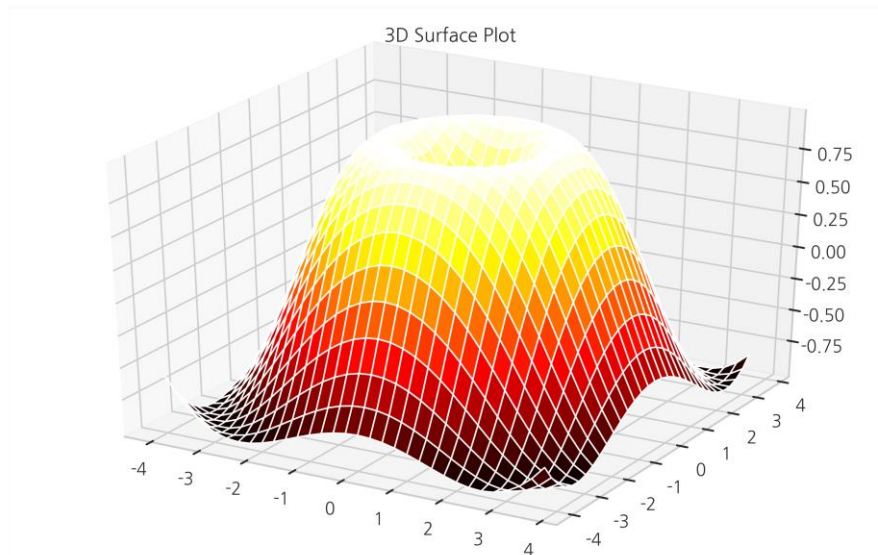
```

```

Y = np.arange(-4, 4, 0.25)
XX, YY = np.meshgrid(X, Y)
RR = np.sqrt(XX**2 + YY**2)
ZZ = np.sin(RR)

fig = plt.figure()
ax = Axes3D(fig)
ax.set_title("3D Surface Plot")
ax.plot_surface(XX, YY, ZZ, rstride=1, cstride=1, cmap='hot')
plt.show()

```



Matplotlib의 triangular grid 사용법

Matplotlib 버전 1.3 부터는 삼각 그리드(triangular grid)에 대한 지원이 추가되었다. 삼각 그리드를 사용하면 기존의 사각형 영역 뿐 아니라 임의의 영역에 대해서 컨투어 플롯이나 서피스 플롯을 그릴 수 있으므로 정의역(domain)이 직사각형이 아닌 2차원 함수도 시각화 할 수 있다.

패키지

삼각 그리드 지원을 위한 코드 중 일부는 tri 서브 패키지 아래에 있으므로 미리 임포트한다.

```
import matplotlib.tri as mtri
```

삼각 그리드 클래스

삼각 그리드 지원을 위한 클래스는 다음과 같다.

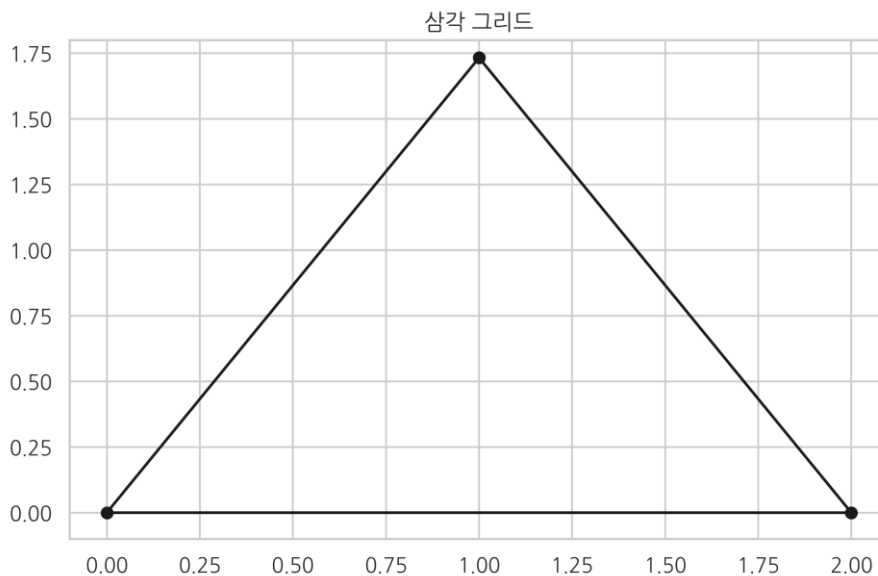
- 삼각 그리드 생성
 - Triangulation
 - http://matplotlib.org/api/tri_api.html?highlight=triangulation#matplotlib.tri.Triangulation
- 삼각 그리드 세분화
 - TriRefiner
 - http://matplotlib.org/api/tri_api.html?highlight=triangulation#matplotlib.tri.TriRefiner
 - UniformTriRefiner
 - http://matplotlib.org/api/tri_api.html?highlight=triangulation#matplotlib.tri.UniformTriRefiner
- 삼각 그리드 플롯
 - triplot
 - http://matplotlib.org/api/pyplot_api.html#matplotlib.pyplot.triplot
 - tricontour
 - http://matplotlib.org/api/pyplot_api.html#matplotlib.pyplot.tricontour
 - tricontourf
 - http://matplotlib.org/api/pyplot_api.html#matplotlib.pyplot.tricontourf
 - tripcolor
 - http://matplotlib.org/api/pyplot_api.html#matplotlib.pyplot.tripcolor
- 삼각 그리드 보간
 - TriInterpolator
 - http://matplotlib.org/api/tri_api.html?highlight=triangulation#matplotlib.tri.TriInterpolator
 - LinearTriInterpolator
 - http://matplotlib.org/api/tri_api.html?highlight=triangulation#matplotlib.tri.LinearTriInterpolator
 - CubicTriInterpolator

- http://matplotlib.org/api/tri_api.html?highlight=triangulation#matplotlib.tri.CubicTriInterpolator

삼각 그리드 생성

삼각 그리드는 Triangulation 클래스로 생성한다. Triangulation 클래스는 x, y, triangles 세 개의 인수를 받는데 x, y는 일련의 점들의 x 좌표와 y 좌표를 나타내는 1 차원 벡터들이고 triangles는 이 점들에 대한 기하학적 위상 정보 즉, 어떤 삼각형이 있으며 각 삼각형이 어떤 점들로 이루어져 있는가를 보인다. 만약 triangles가 주어지지 않으면 자동으로 생성한다.

```
x = np.array([0, 1, 2])
y = np.array([0, np.sqrt(3), 0])
triangles = [[0, 1, 2]]
triang = mtri.Triangulation(x, y, triangles)
plt.title("삼각 그리드")
plt.triplot(triang, 'ko-')
plt.xlim(-0.1, 2.1)
plt.ylim(-0.1, 1.8)
plt.show()
```

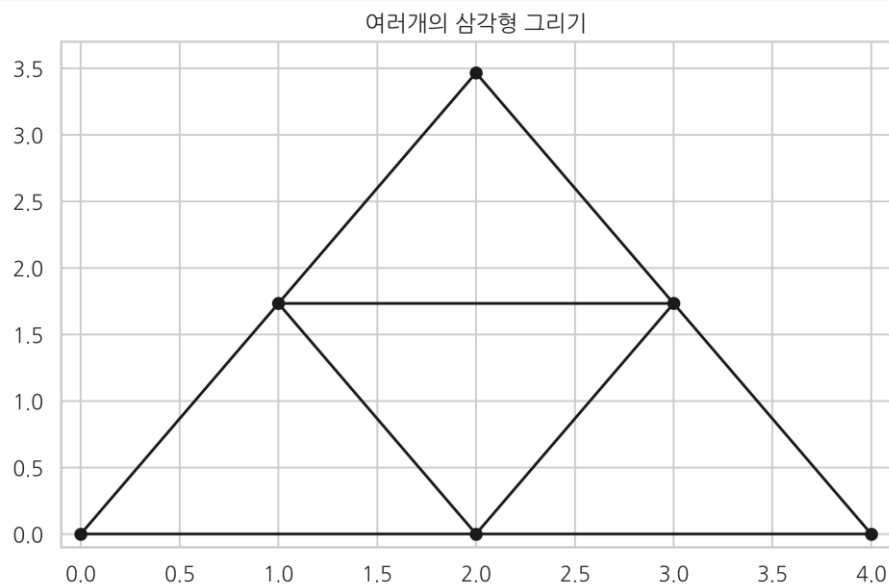


```
x = np.asarray([0, 1, 2, 3, 4, 2])
y = np.asarray([0, np.sqrt(3), 0, np.sqrt(3), 0, 2*np.sqrt(3)])
```

```

triangles = [[0, 1, 2], [2, 3, 4], [1, 2, 3], [1, 3, 5]]
triang = mtri.Triangulation(x, y, triangles)
plt.title("여러개의 삼각형 그리기")
plt.triplot(triang, 'ko-')
plt.xlim(-0.1, 4.1)
plt.ylim(-0.1, 3.7)
plt.show()

```



그리드 세분화

그리드를 더 세분화하려면 TriRefiner 또는 UniformTriRefiner 를 사용한다. 이 클래스들은 다음과 같은 메서드를 가진다.

- refine_triangulation : 단순히 삼각 그리드를 세분화
 - http://matplotlib.org/api/tri_api.html#matplotlib.tri.UniformTriRefiner.refine_triangulation
- refine_field : 실제 함수 값에 대해 최적화된 삼각 그리드 생성
 - http://matplotlib.org/api/tri_api.html#matplotlib.tri.UniformTriRefiner.refine_field

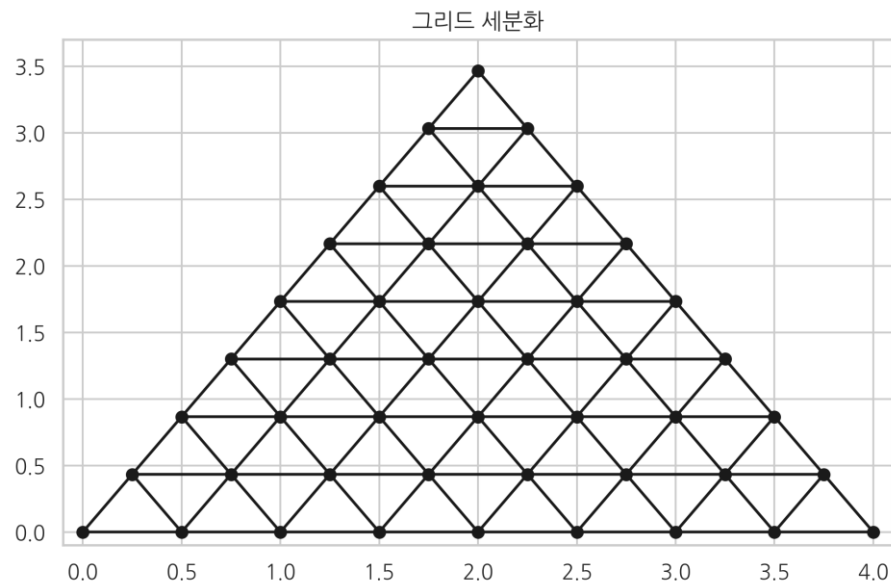
```

refiner = mtri.UniformTriRefiner(triang)
triang2 = refiner.refine_triangulation(subdiv=2)
plt.title("그리드 세분화")

```



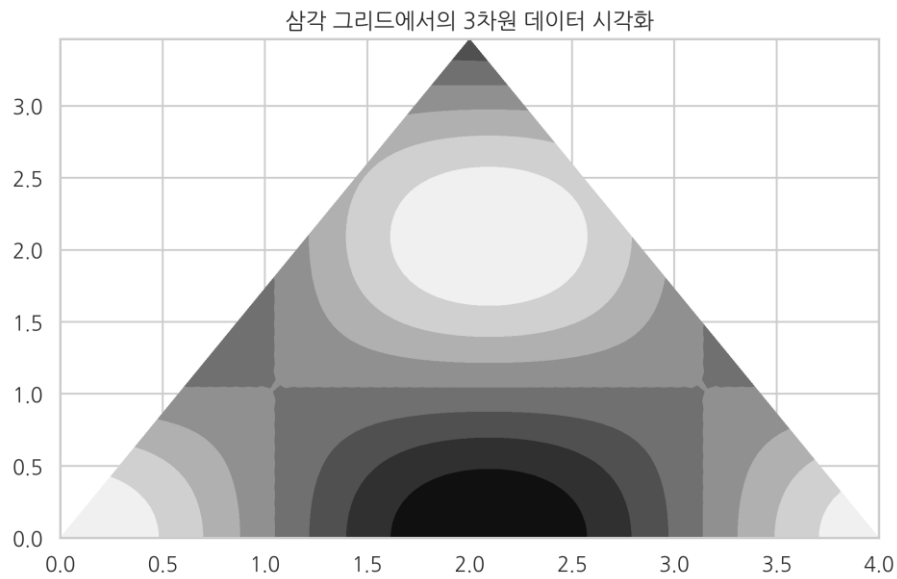
```
plt.triplot(triang2, 'ko-')
plt.xlim(-0.1, 4.1)
plt.ylim(-0.1, 3.7)
plt.show()
```



그리드 플롯

이렇게 만들어진 그리드상에 `tricontour`, `tricontourf`, `plot_trisurf`, `plot_trisurf` 등의 명령을 사용하여 2 차원 등고선(contour) 플롯이나 3 차원 표면(surface) 플롯을 그릴 수 있다.

```
triang5 = refiner.refine_triangulation(subdiv=5)
z5 = np.cos(1.5*triang5.x)*np.cos(1.5*triang5.y)
plt.title("삼각 그리드에서의 3 차원 데이터 시각화")
plt.tricontourf(triang5, z5, cmap="gray")
plt.show()
```



```
from mpl_toolkits.mplot3d import Axes3D
from matplotlib import cm

triang3 = refiner.refine_triangulation(subdiv=3)
z3 = np.cos(1.5 * triang3.x) * np.cos(1.5 * triang3.y)

fig = plt.figure()
ax = fig.gca(projection='3d')
ax.set_title("삼각 그리드에서의 3D Surface Plot")
ax.plot_trisurf(triang3.x, triang3.y, z3, cmap=cm.jet, linewidth=0.2)
ax.tricontourf(triang3, z3, zdir='z', offset=-1.2, cmap=cm.coolwarm)
ax.set_zlim(-1, 1)
ax.view_init(40, -40)
plt.show()
```

삼각 그리드에서의 3D Surface Plot

